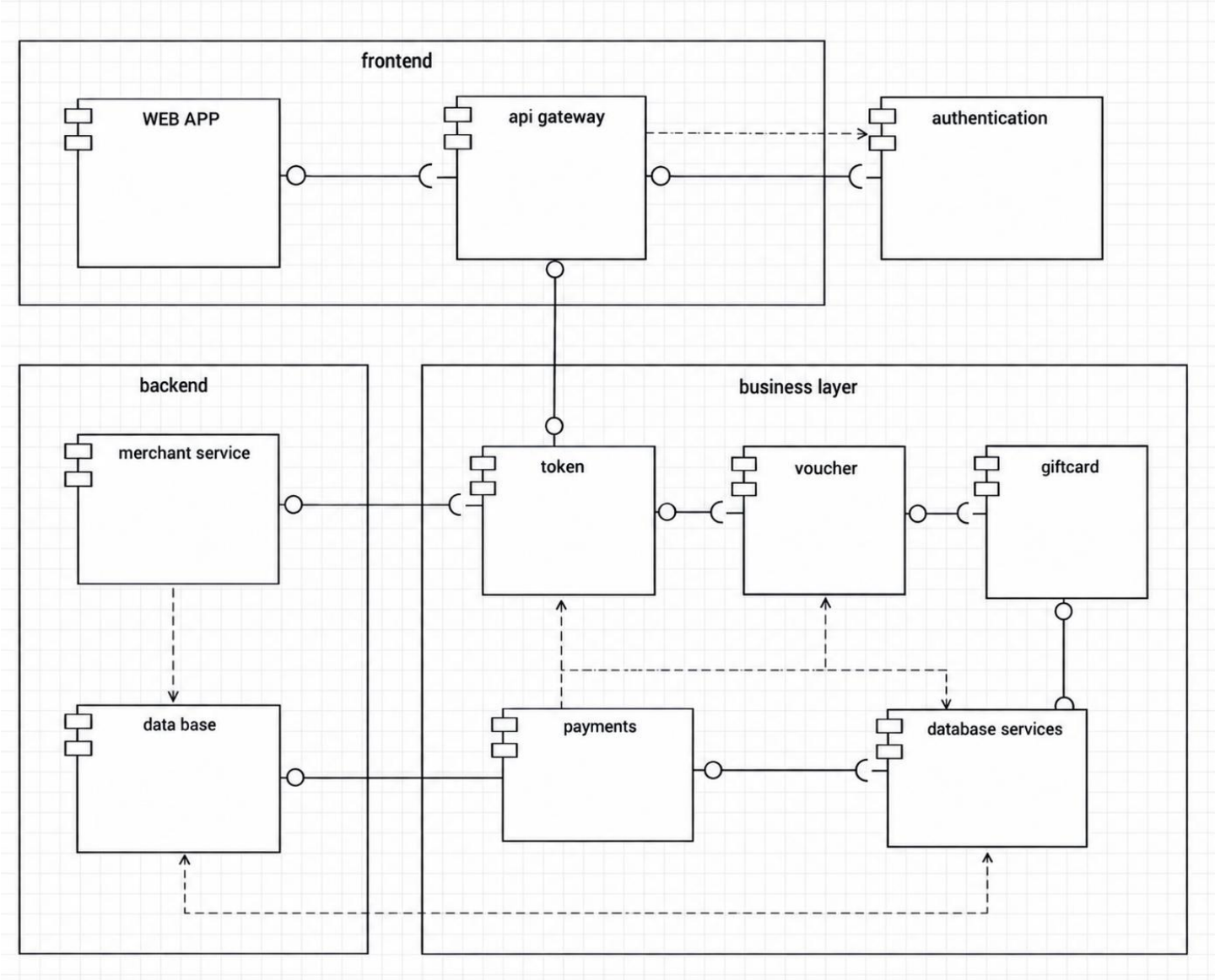
Lab 3: Component Modelling & Architectural Pattern Selection

Problem Statement: Digital Gift Card & Loyalty Point Exchange



1. Scenario Review

• **Functional requirements:** validate loyalty points; display points and token value; convert eligible points using the current dynamic rate; redeem gift cards using exchange tokens; generate unique digital vouchers; lock vouchers to prevent duplicate redemption.

• **Non-functional requirements:** conversion and redemption within 250 ms; zero ledger balance drift; single-use vouchers; invalid/expired points must not be converted; unauthorized users must be rejected.

• **Main challenges:** transaction accuracy, voucher anti-fraud protection, secure access to user assets, dynamic conversion rates, and low-latency conversion/redemption.

2. Architectural Style Analysis

Architecture	Structure	Advantages for this system	Limitations
Layered	Presentation, business logic, and data/backend responsibilities are separated into layers.	Clear separation of concerns; easier maintenance; business rules such as token conversion and voucher locking stay separated from UI and storage.	Cross-layer communication can add overhead; tight dependency between layers can make independent scaling harder.
Microservices	Small, independently deployable services.	Independent scaling, fault isolation, and independent deployment of services such as voucher or token processing.	Higher operational complexity, network latency, and data consistency challenges.
Client-Server	Clients communicate with a centralized server.	Centralized control and simpler data consistency.	Central server can become a scalability bottleneck and single point of failure.

Selected architecture: Layered Architecture. The submitted diagram reflects a frontend layer, backend layer, and business layer, with interfaces and dependencies connecting the relevant components.

3. Written Architectural Justification

Architecture Selection: We chose **Layered Architecture** for the Digital Gift Card & Loyalty Point Exchange System.

Architectural Choice

The system is organized into a frontend layer, backend layer, and business layer. This separates user interaction and API handling from merchant integration, token conversion, voucher processing, gift-card redemption, payment processing, and database services.

Reason 1 – Separation of responsibilities

The system contains distinct responsibilities such as loyalty-point validation, dynamic point-to-token conversion, voucher locking, and gift-card redemption. Keeping these responsibilities in separate business components makes the design easier to understand, maintain, and modify without changing the user interface.

Reason 2 – Data and transaction control

The system requires zero ledger balance drift and single-use vouchers. A layered design allows business rules and transaction-related processing to be separated from database access, making it clearer where validation, balance updates, and voucher controls are handled.

Security Advantage

The authentication component is separated from the application-facing components. The API Gateway can route requests through authentication and authorization before protected loyalty points, tokens, and vouchers are accessed. This supports the requirement that only authorized Account Holders can access their assets.

Performance Benefit

Separating the frontend/API handling from business processing and data services allows frequently used operations to be handled by focused components. This supports the requirement that point conversion and gift-card redemption should complete within 250 ms, subject to implementation and infrastructure performance.

Lab 3 UML compliance: The diagram uses component notation, provided/required interfaces, assembly connectors, and dependency relationships as specified in the lab instructions.